

Contents

RL-Based Value Function Approximation for Sequential Genetic Testing	2
## Full Mathematical Presentation	2
SLIDE 1 — Problem Statement	2
SLIDE 2 — Genetic Model	2
2.1 Population Prior (Hardy-Weinberg Equilibrium)	2
2.2 Mendelian Inheritance CPD	3
2.3 Independent Assortment (Two Genes)	3
SLIDE 3 — Belief State & Bayesian Update	3
3.1 State and Belief	3
3.2 Belief Update	3
3.3 Marginal Belief for Person i	3
3.4 Factorised Two-Gene Belief (Efficient Computation)	4
SLIDE 4 — Reward Model	4
4.1 Stopping Reward	4
4.2 Testing Reward (Immediate)	4
SLIDE 5 — POMDP Formulation	5
5.1 Formal Definition	5
5.2 Q-Function	5
SLIDE 6 — Exact Optimal Value Function (V^*)	6
6.1 Bellman Optimality Equation	6
6.2 Backward Induction (Exact DP)	6
6.3 V_{stop} at Root (Evaluation Anchor)	6
SLIDE 7 — Evaluation Metric: ratio2	6
7.1 Definition	6
7.2 Interpretation	7
7.3 Computing L (True Policy Value, No Approximation)	7
SLIDE 8 — Step 1: Proof of Concept (Single Gene)	7
SLIDE 9 — Step 2: Sanity Checks (15/15 Pass)	7
SLIDE 10 — Step 3: Two-Gene Extension	8
SLIDE 11 — Step 4: Multi-Family MLP	8
11.1 Architecture	8
11.2 Training	8
SLIDE 12 — Step 4: Results	9
12.1 Full Results Table	9
12.2 Interpretation	9
12.3 Myopic Baseline	9
SLIDE 13 — ADP Baseline (ABCD-16)	10
13.1 Approximate Linear Program Formulation	10
13.2 ADP Greedy Policy	10
SLIDE 14 — Step 5: Graph Neural Network	10
14.1 Motivation	10
14.2 Graph Representation	10
14.3 Message Passing (2 Rounds)	11
14.4 Readout	11
14.5 Batched Training (Same-Topology Optimisation)	11
14.6 Training Setup	12

SLIDE 15 — GNN Results	12
15.1 Training Convergence	12
15.2 ratio2 Comparison: GNN vs MLP	12
SLIDE 16 — Model Comparison	13
SLIDE 17 — Limitations & Honest Caveats	13
SLIDE 18 — Summary	14
APPENDIX A — Notation Reference	14
APPENDIX B — Pedigree Structures	15
APPENDIX C — Answers to Anticipated Questions	15

RL-Based Value Function Approximation for Sequential Genetic Testing

Full Mathematical Presentation

SLIDE 1 — Problem Statement

Clinical setting: A family pedigree of N individuals may carry a hereditary disease mutation. Each person can be genetically tested. Testing costs money. Untested carriers incur future clinical costs. The goal is an optimal sequential testing policy: *who to test next, and when to stop*.

Two disease genes: GeneA and GeneB. Each person's genotype per gene is $g \in \{0, 1, 2\}$ where: - 0 = NN (non-carrier, two normal alleles) - 1 = ND (heterozygous carrier, one disease allele) - 2 = DD (homozygous affected, two disease alleles)

Two family structures studied: | Family | N (people) | Structure | |-----|-----|-----|
-----| | Extended | 6 | Grandparents + parents + proband + sibling | | ThreeGeneration | 5 |
Grandparent + parents + proband + sibling |

Two allele frequency regimes: | Regime | $p(\text{GeneA})$ | $p(\text{GeneB})$ | |-----|-----|-----| | LowHigh
| 0.02 | 0.15 | | MediumEven | 0.08 | 0.08 |

Two cost presets: | Preset | a | b | δ | c_{fixed} | c_{var} | |-----|-----|-----|-----| | Base |
-0.08 | -0.04 | 0.60 | 0.01 | 0.02 | | Aggressive | -0.12 | -0.06 | 0.70 | 0.01 | 0.02 |

2 families $\times$ 2 regimes $\times$ 2 presets = **8 canonical problem instances**.

SLIDE 2 — Genetic Model

2.1 Population Prior (Hardy-Weinberg Equilibrium)

For a founder (no parents in the pedigree), the prior distribution over genotype g given population allele frequency p is:

$$\begin{aligned}
 P(g = 0) &= (1 - p)^2 && \text{(NN: two normal alleles)} \\
 P(g = 1) &= 2p(1 - p) && \text{(ND: heterozygous carrier)} \\
 P(g = 2) &= p^2 && \text{(DD: homozygous affected)}
 \end{aligned}$$

2.2 Mendelian Inheritance CPD

For a child with father genotype g_f and mother genotype g_m , let the probability that each parent passes the disease allele D be:

$$\begin{aligned} f_pass(g_f) &= 0 && \text{if } g_f = 0 \\ &= 0.5 && \text{if } g_f = 1 \\ &= 1 && \text{if } g_f = 2 \end{aligned}$$

Then the child's conditional probability distribution is:

$$P(g_child = 0 \mid g_f, g_m) = (1 - f_pass(g_f)) \cdot (1 - f_pass(g_m)) \quad (NN)$$

$$P(g_child = 2 \mid g_f, g_m) = f_pass(g_f) \cdot f_pass(g_m) \quad (DD)$$

$$P(g_child = 1 \mid g_f, g_m) = 1 - P(g_child=0) - P(g_child=2) \quad (ND)$$

This is a 3×9 conditional probability table (3 child outcomes $\times$ 9 parent combinations).

2.3 Independent Assortment (Two Genes)

GeneA and GeneB are assumed to segregate independently (non-linked, on separate chromosomes). The joint prior and CPDs factorize:

$$P(g_A, g_B \mid \text{person}) = P(g_A \mid \text{person}) \cdot P(g_B \mid \text{person})$$

This factorization is computed per-gene independently, then combined. It reduces the state space from $O(9^N)$ joint states to $O(3^N)$ per gene.

SLIDE 3 — Belief State & Bayesian Update

3.1 State and Belief

Let $X = (X_1^A, X_1^B, \dots, X_N^A, X_N^B)$ denote the unknown genotypes of all N individuals for both genes. The observation set after some tests is:

$$O = \{(i_1, o_1), (i_2, o_2), \dots, (i_k, o_k)\}$$

where $o_j \in \{0,1,2\}^2$ is the observed joint genotype of person i_j .

The **belief state** is the Bayesian posterior:

$$b_0(x) = P(X = x \mid O)$$

3.2 Belief Update

After observing test result (i, g) , the posterior updates by conditioning:

$$P(X = x \mid O \cup \{(i^*, g^*)\}) \propto P(X = x \mid O) \cdot 1[x_{\{i^*\}} = g^*]$$

followed by renormalization. In practice, computed via Bayesian network message passing over the pedigree graph.

3.3 Marginal Belief for Person i

The marginal posterior for person i over gene k is:

$$b_0^k(i, g) = P(X_i^k = g \mid 0) \quad \text{for } g \in \{0, 1, 2\}$$

The **carrier probability** (genotype 1 or 2) for person i under gene k :

$$p_i^k = b_0^k(i, 1) + b_0^k(i, 2) = P(X_i^k \in \{1, 2\} \mid 0)$$

3.4 Factorised Two-Gene Belief (Efficient Computation)

For each gene $k \in \{A, B\}$ independently: 1. Build full joint $P(X^k \mid O^k)$ over all 3^N genotype combinations via forward-backward on the pedigree Bayesian network (O^k = test results for gene k only). 2. Marginalise to get $b_0^k(i, \cdot)$ for each person i .

Combined joint for person i under independent assortment:

$$P(X_i^A = g_A, X_i^B = g_B \mid 0) = b_0^A(i, g_A) \cdot b_0^B(i, g_B)$$

Complexity: $O(3^N)$ per gene vs $O(9^N)$ for the full joint — a 3^N factor improvement.

SLIDE 4 — Reward Model

4.1 Stopping Reward

When stopping at state O (having tested the individuals in O), the reward for leaving person $i \notin \text{tested}(O)$ untested is:

For each gene $k \in \{A, B\}$:

$$r_{\text{stop}}(i, b_0, k) = a_k \cdot (p_i^k - \delta_k \cdot (p_i^k)^2) + b_k \cdot (p_i^k - (p_i^k)^2)$$

where: - p_i^k = carrier probability of person i for gene k - a_k = carrier detection value coefficient (negative $\rightarrow$ cost of missing a carrier) - b_k = heterozygote-specific adjustment coefficient - δ_k = homozygosity discount ($\delta \in [0, 1]$; reduces value for homozygous carriers)

The term $(p_i^k - \delta_k \cdot (p_i^k)^2)$ captures the expected value of person i being a carrier, discounted by the probability of being homozygous (which is already clinically managed differently). The term $(p_i^k - (p_i^k)^2) = \text{Var}[X_i^k \in \{1, 2\}]$ is the Bernoulli variance — penalising uncertainty.

Total stopping reward for person i across both genes:

$$r_{\text{stop}}(i, b_0) = \sum_{k \in \{A, B\}} r_{\text{stop}}(i, b_0, k) + c_{\text{base}}(i)$$

($c_{\text{base}} = 0$ in our experiments.)

Total V_{stop} at state O :

$$V_{\text{stop}}(O) = \sum_{i \notin \text{tested}(O)} r_{\text{stop}}(i, b_0)$$

4.2 Testing Reward (Immediate)

The immediate reward for testing person i at state O :

$$r_{\text{test}}(i, b_0) = \sum_{k \in \{A, B\}} [a_k \cdot (1 - \delta_k) \cdot p_i^k] - c_{\text{fixed}} - c_{\text{var}} \cdot P(\text{any positive} \mid i, b_0)$$

where:

$$P(\text{any positive} \mid i, b_0) = 1 - \prod_{k \in \{A,B\}} (1 - p_i^k)$$

This is the probability that at least one gene shows a positive result for person i .

Interpretation: $-a_k \cdot (1 - \delta_k) \cdot p_i^k$ = expected reward from confirming a carrier (not homozygous)

$-c_{\text{fixed}}$ = fixed cost of the test (e.g., lab setup) $-c_{\text{var}} \cdot P(\text{any positive})$ = variable cost, paid only when the test returns positive

SLIDE 5 — POMDP Formulation

5.1 Formal Definition

$$\text{POMDP} = (S, A, O, T, R, b_0)$$

State space S :

$$x = (x_1^A, x_1^B, \dots, x_N^A, x_N^B) \in \{0,1,2\}^{2N}$$

Hidden — never directly observed.

Observation set (sufficient statistic):

$$O \subseteq \{(i, g) : i \in \{1, \dots, N\}, g \in \{0,1,2\}^2\}$$

Monotonically growing — tests are irreversible.

Belief state:

$$b_0 : \{0,1,2\}^{2N} \rightarrow [0,1], \quad \sum_x b_0(x) = 1$$

Action space $A(O)$:

$$A(O) = \{\text{STOP}\} \cup \{i : i \notin \text{tested}(O)\}$$

Transition (belief update): After testing person i with outcome o :

$$b_{\{O \cup \{(i,o)\}\}}(x) \propto b_0(x) \cdot 1[x_i = o]$$

Observation probability (marginal from belief):

$$P(\text{outcome} = o \mid \text{test } i, \text{belief } b_0) = b_0^A(i, o_A) \cdot b_0^B(i, o_B)$$

Reward:

$$R(O, \text{STOP}) = V_{\text{stop}}(O) = \sum_{\{i \notin \text{tested}(O)\}} r_{\text{stop}}(i, b_0)$$

$$R(O, \text{test } i) = r_{\text{test}}(i, b_0) \quad [\text{immediate only; future reward counted in next state}]$$

Initial belief b_0 : Hardy-Weinberg priors on founders, Mendelian inheritance on children. No tests have been performed: $O_0 = \square$.

5.2 Q-Function

The action-value function (Q-function) at state O :

$$Q^*(O, \text{STOP}) = V_{\text{stop}}(O)$$

$$Q^*(O, \text{test } i) = r_{\text{test}}(i, b_O) + \sum_{o \in \{0,1,2\}^2} P(o | i, b_O) \cdot V^*(O \cup \{(i,o)\})$$

SLIDE 6 — Exact Optimal Value Function (V^*)

6.1 Bellman Optimality Equation

The optimal value function satisfies:

$$V^*(O) = \max(\\ V_{\text{stop}}(O), \\ \max_{i \notin \text{tested}(O)} Q^*(O, \text{test } i) \\)$$

with boundary condition:

$$V^*(O) = 0 \quad \text{when } \text{tested}(O) = \{1, \dots, N\} \quad (\text{all individuals tested})$$

Note: at the fully-tested terminal state, no stopping reward is counted (all rewards were claimed at the point of stopping or along the path).

6.2 Backward Induction (Exact DP)

Because the state O grows monotonically (tests are irreversible), the belief MDP is **acyclic**. We solve it exactly by backward induction:

Algorithm: Backward Induction

1. Enumerate all reachable states O , sorted by $|O|$ descending (most-tested first)
2. For each state O in reverse order:
 - a. Compute $V_{\text{stop}}(O) = \sum_{i \notin \text{tested}} r_{\text{stop}}(i, b_O)$
 - b. For each $i \notin \text{tested}(O)$:
$$Q_i = r_{\text{test}}(i, b_O) + \sum_o P(o|i, b_O) \cdot V^*(O \cup \{(i,o)\})$$
 - c. $V^*(O) = \max(V_{\text{stop}}(O), \max_i Q_i)$
 - d. $\pi^*(O) = \text{argmax over } \{\text{STOP}, \text{test}_i\}$

Complexity: - States: ThreeGeneration $\sim 20,816$; Extended $\sim 107,728$ (two genes) - Per state: N action evaluations, each summing over $|\{0,1,2\}^2| = 9$ outcomes - Total: $O(|S| \cdot N \cdot 9)$

6.3 V_{stop} at Root (Evaluation Anchor)

$$V_{\text{stop}}(O_0) = \sum_{i=1}^N r_{\text{stop}}(i, b_0)$$

This is the value of never testing anyone. Used as the denominator anchor in ratio2.

SLIDE 7 — Evaluation Metric: ratio2

7.1 Definition

Let: - $V^*(O_0)$ = exact optimal value from root state (backward induction) - L = true expected value of the net's greedy policy $\hat{\pi}$ from O_0 - $V_{\text{stop}}(O_0)$ = value of stopping immediately (test nobody)

$$\text{ratio2} = (V^*(O_0) - L) / (V^*(O_0) - V_{\text{stop}}(O_0))$$

7.2 Interpretation

$\text{ratio2} = 0 \Leftrightarrow L = V^*(O_0)$ [policy is perfectly optimal]

$\text{ratio2} = 1 \Leftrightarrow L = V_{\text{stop}}$ [policy is as bad as stopping immediately]

A ratio2 of 0.05 means the policy captures **95% of the achievable improvement** over stopping immediately.

7.3 Computing L (True Policy Value, No Approximation)

Given approximate value function $\hat{V}_\theta$, the greedy policy is:

$$\pi(0) = \operatorname{argmax}_{\{a \in A(0)\}} \hat{Q}(0, a)$$

$$\hat{Q}(0, \text{STOP}) = V_{\text{stop}}(0)$$

$$\hat{Q}(0, \text{test } i) = r_{\text{test}}(i, b_0) + \sum_o P(o|i, b_0) \cdot \hat{V}_\theta(0 \cup \{(i, o)\})$$

The **true policy value** L is computed by exact recursion:

$$V^{\{\pi\}}(0) = r_{\text{test}}(\pi(0), b_0) + \sum_o P(o|\pi(0), b_0) \cdot V^{\{\pi\}}(0 \cup \{(\pi(0), o)\})$$

if $\pi(0) \neq \text{STOP}$

$$V^{\{\pi\}}(0) = V_{\text{stop}}(0) \quad \text{if } \pi(0) = \text{STOP}$$

with $L = V^{\{\hat{\pi}\}}(O_0)$. We never substitute $\hat{V}$ for V^* in the evaluation — L is exact.

SLIDE 8 — Step 1: Proof of Concept (Single Gene)

Setup: - 1 gene (GeneA), ThreeGeneration family (N=5) - State vector: $x_s = [P(g=0|i), P(g=1|i), P(g=2|i)]_{i=1}^5 \in \mathbb{R}^{\{15\}}$ - 2,752 reachable belief states

Architecture: MLP(15 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 1) Each hidden layer: $h^{\{l\}} = \text{ReLU}(W^{\{l\}} h^{\{l-1\}} + b^{\{l\}})$ Output: $\hat{V}_\theta(s) = W^{\{\text{out}\}} h^{\{L\}} + b^{\{\text{out}\}}$ (linear, no activation)

Loss:

$$L(\theta) = (1/|S_{\text{train}}|) \sum_{\{s \in S_{\text{train}}\}} (\hat{V}_\theta(s) - V^*(s))^2$$

Optimiser: Adam (lr = 1×10^{-3}), 500 epochs, batch size 512

Result: $\hat{V}_\theta(s) \approx V(s)$ — the MLP successfully approximates the value function from the flat belief vector. Confirmed that the belief vector contains sufficient information to recover V.

SLIDE 9 — Step 2: Sanity Checks (15/15 Pass)

Before training anything on V^* , we verified the backward induction is correct. 15 tests across 3 datasets (ThreeGeneration, Extended, two-gene):

ID	Test	Formal condition
T1	Optimality lower bound	$V^*(O) \geq V_{\text{stop}}(O) \forall O$
T2	Monotonicity	$V(O) \geq V(O')$ when $O \subseteq O'$
T3	Terminal value	$V^*(O) = 0$ when $\text{tested}(O) = \{1, \dots, N\}$
T4	Root is maximum	$V(\square) = \max_O V(O)$
T5	Test value bound	$r_{\text{test}}(i, b_O) < V^*(O) - V_{\text{stop}}(O) \forall O, i$

All 15 tests pass. The exact DP is verified correct.

SLIDE 10 — Step 3: Two-Gene Extension

State vector (two genes, N=5):

$$\mathbf{x}_s = [P(gA=0|i), P(gA=1|i), P(gA=2|i), P(gB=0|i), P(gB=1|i), P(gB=2|i)]_{i=1}^5 \in \mathbb{R}^{\{30\}}$$

6 probabilities per person $\times$ 5 people = 30 dimensions.

Architecture: $\text{MLP}(30 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 1)$

Technical challenge: Computing the two-gene joint posterior naively requires enumerating $| \{0,1,2\}^2 |^N = 9^5 = 59,049$ joint states per BFS step — process killed.

Solution: Factorised belief computation using independent assortment: 1. Run single-gene backward induction for GeneA: $O(3^N)$ per state 2. Run single-gene backward induction for GeneB: $O(3^N)$ per state 3. Combine: $P(gA, gB | i, O) = P(gA | i, O) \cdot P(gB | i, O)$

State space: 20,816 reachable belief states (both genes). Training: same as Step 1.

SLIDE 11 — Step 4: Multi-Family MLP

11.1 Architecture

Input: $\mathbf{x}_s \in \mathbb{R}^{\{36\}}$ — belief vector, zero-padded to max family size (6 people $\times$ 6 probs). ThreeGeneration (5 people, 30-dim) is zero-padded: $\mathbf{x}_s = [\mathbf{x}_s^{\{30\}}, 0^6] \in \mathbb{R}^{\{36\}}$.

$$\text{MLPValueNet: } \mathbb{R}^{\{36\}} \rightarrow \mathbb{R}^{\{128\}} \rightarrow \mathbb{R}^{\{64\}} \rightarrow \mathbb{R}^{\{32\}} \rightarrow \mathbb{R}$$

$$\begin{aligned} h^{\{1\}} &= \text{ReLU}(W^{\{1\}} \mathbf{x}_s + b^{\{1\}}), & W^{\{1\}} &\in \mathbb{R}^{\{128 \times 36\}} \\ h^{\{2\}} &= \text{ReLU}(W^{\{2\}} h^{\{1\}} + b^{\{2\}}), & W^{\{2\}} &\in \mathbb{R}^{\{64 \times 128\}} \\ h^{\{3\}} &= \text{ReLU}(W^{\{3\}} h^{\{2\}} + b^{\{3\}}), & W^{\{3\}} &\in \mathbb{R}^{\{32 \times 64\}} \\ \hat{V}_\theta(s) &= W^{\{4\}} h^{\{3\}} + b^{\{4\}}, & W^{\{4\}} &\in \mathbb{R}^{\{1 \times 32\}} \end{aligned}$$

Total parameters: $36 \times 128 + 128 + 128 \times 64 + 64 + 64 \times 32 + 32 + 32 \times 1 + 1 = \mathbf{15,105}$

11.2 Training

Train set: 6 families (all Extended variants + ThreeGeneration $\times$ LowHigh) **Test set:** 2 families (ThreeGeneration $\times$ MediumEven) — **never seen during training**

Total training states: 472,544 belief states (4×107,728 Extended + 2×20,816 ThreeGeneration)
 Epochs: 800
 Batch size: 512
 Learning rate: 1×10^{-3} (Adam, default $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$)
 Final train loss: 6.99×10^{-4}

Loss function:

$$L(\theta) = (1/|S_{\text{train}}|) \sum_{\{s \in S_{\text{train}}\}} (\hat{V}_{\theta}(s) - V^*(s))^2$$

Checkpointing: every 50 epochs, save {epoch, θ , optimizer state, loss history} atomically (write to .tmp, then rename) for SLURM fault tolerance.

SLIDE 12 — Step 4: Results

12.1 Full Results Table

Family	Split	$V^*(\emptyset)$	$V_{\text{stop}}(\emptyset)$	$L(\text{net})$	ratio2(net)	Baseline	xBetter
Extended_LowHigh_Base	TRAIN	-0.1668	-0.2802	-0.1710	0.0370	0.1007 (ADP)	2.7×
Extended_LowHigh_Aggressive	TRAIN	-0.1788	-0.4119	-0.4119	-0.1832	0.0190	—
—							
Extended_MediumEven_Base	TRAIN	-0.1620	-0.2944	-0.1684	0.0484	0.1590 (ADP)	3.3×
Extended_MediumEven_Aggressive	TRAIN	-0.1738	-0.4365	-0.4365	-0.1810	0.0273	—
—							
ThreeGeneration_LowHigh_Base	TRAIN	-0.1372	-0.2180	-0.2180	-0.1390	0.0224	—
—							
ThreeGeneration_LowHigh_Aggressive	TRAIN	-0.1464	-0.3203	-0.3203	-0.1473	0.0051	0.0513 (ADP) 10.0×
ThreeGeneration_MediumEven_Base	TEST	-0.1332	-0.2290	-0.1371	0.0398	0.0680 (myopic)	1.7×
ThreeGeneration_MediumEven_Aggressive	TEST	-0.1422	-0.3395	-0.1467	0.0232	0.0390 (myopic)	1.7×

12.2 Interpretation

- Net captures **96-98% of the achievable improvement** over stopping on test families
- On 3 families with published ADP baselines: **net beats ADP by 2.7-10×**
- ADP baseline note: Only 3 of 8 families have published ADP baselines (from expected.json). Test families use myopic baseline (see §12.3) because full Gurobi license unavailable.

12.3 Myopic Baseline

The one-step greedy policy: at each state, test the person maximising immediate expected reward plus expected stopping value of the next state:

$$\pi_{\text{myopic}}(0) = \operatorname{argmax}_{\{i \notin \text{tested}(0)\}} [r_{\text{test}}(i, b_0) + \sum_o P(o|i, b_0) \cdot V_{\text{stop}}(0 \cup \{(i, o)\})]$$

This is strictly weaker than ADP (myopic sees one step; ADP uses an approximate long-horizon value function). Therefore net beating myopic by 1.7× is a **conservative lower bound** on the improvement

over ADP.

SLIDE 13 — ADP Baseline (ABCD-16)

13.1 Approximate Linear Program Formulation

The ABCD-16 ADP constructs a 16-dimensional feature map $\phi: S \rightarrow \mathbb{R}^{16}$ using hand-crafted domain features (carrier mass on frontier nodes, bridge depth mass, myopic test margin, etc.) and solves the Approximate Linear Program (ALP):

$$\min_{\{\theta \in \mathbb{R}^{16}\}} \sum_{s \in S} \rho(s) \cdot \theta^T \phi(s)$$

subject to:

$$\begin{aligned} \theta^T \phi(s) &\geq V_{\text{stop}}(s) & \forall s \in S \\ \theta^T \phi(s) &\geq r_{\text{test}}(i, s) + \sum_o P(o|i, s) \cdot \theta^T \phi(s') & \forall s \in S, \forall i \notin \text{tested}(s) \end{aligned}$$

where $\rho(s)$ is a state relevance weighting (uniform in this implementation) and $s' = s \cup \{(i, o)\}$ is the successor state.

These are the **Bellman inequality constraints**: the approximate value function $\theta^T \phi(s)$ must be an upper bound on $V^*(s)$ everywhere (dual LP / LP relaxation of DP).

The second constraint rearranges to:

$$[\phi(s) - \sum_o P(o|i, s) \phi(s')] \cdot \theta \geq r_{\text{test}}(i, s)$$

Solved with Gurobi (LP, ~5-30 min per family). Requires **re-solving per family**: new allele frequencies, new cost parameters, or new family structure $\rightarrow$ new LP.

13.2 ADP Greedy Policy

$$\pi_{\text{ADP}}(0) = \operatorname{argmax}_{\{a \in A(0)\}} Q_{\text{ADP}}(0, a)$$

$$Q_{\text{ADP}}(0, \text{STOP}) = V_{\text{stop}}(0)$$

$$Q_{\text{ADP}}(0, \text{test } i) = r_{\text{test}}(i, b_0) + \sum_o P(o|i, b_0) \cdot \theta^{*T} \phi(0 \cup \{(i, o)\})$$

where θ^* is the LP solution.

SLIDE 14 — Step 5: Graph Neural Network

14.1 Motivation

The MLP treats the belief vector as a flat 36-dim input. It ignores structural roles: a grandparent's belief influences children differently from a sibling's belief. A GNN explicitly encodes who is parent of whom via directed edges.

14.2 Graph Representation

For state O with belief b_O :

Node features (7 per node/person):

$$h_i^{\{0\}} = [P(gA=0|i,0), P(gA=1|i,0), P(gA=2|i,0), \\ P(gB=0|i,0), P(gB=1|i,0), P(gB=2|i,0), \\ 1_{\{i \in \text{tested}(0)\}}] \in \mathbb{R}^7$$

Edge set: directed parent $\rightarrow$ child (pedigree structure)

$E = \{(\text{parent}, \text{child}) : \text{child has parent as biological parent in pedigree}\}$

ThreeGeneration: $N=5$ nodes, $|E|=4$ edges. Extended: $N=6$ nodes, $|E|=6$ edges.

14.3 Message Passing (2 Rounds)

Architecture: PedigreeGNN

node_feat_dim = 7, hidden_dim = 32, n_rounds = 2

At each round $t \in \{1, 2\}$:

Message function:

$$m_{\{j \rightarrow i\}}^{\{t\}} = \text{MLP_msg}^{\{t\}}([h_j^{\{t-1\}}; h_i^{\{t-1\}}]) \\ = \text{ReLU}(W_{\text{msg}}^{\{t\}}[h_j^{\{t-1\}}; h_i^{\{t-1\}}] + b_{\text{msg}}^{\{t\}})$$

where $[:,]$ denotes concatenation.

Aggregation (mean over in-neighbours):

$$\text{agg}_i^{\{t\}} = (1/|N(i)|) \sum_{j \in N(i)} m_{\{j \rightarrow i\}}^{\{t\}}$$

$|N(i)|$ = number of parents of i ; $|N(i)| = 0$ for founders ($\rightarrow \text{agg} = 0$)

Update function:

$$h_i^{\{t\}} = \text{MLP_upd}^{\{t\}}([h_i^{\{t-1\}}; \text{agg}_i^{\{t\}}]) \\ = \text{ReLU}(W_{\text{upd}}^{\{t\}}[h_i^{\{t-1\}}; \text{agg}_i^{\{t\}}] + b_{\text{upd}}^{\{t\}})$$

Layer dimensions:

Round 1: $W_{\text{msg}}^{\{1\}} \in \mathbb{R}^{\{32 \times 14\}}$, $W_{\text{upd}}^{\{1\}} \in \mathbb{R}^{\{32 \times 39\}}$ (7+32=39 input)

Round 2: $W_{\text{msg}}^{\{2\}} \in \mathbb{R}^{\{32 \times 64\}}$, $W_{\text{upd}}^{\{2\}} \in \mathbb{R}^{\{32 \times 64\}}$ (32+32=64 input)

14.4 Readout

Global mean pooling over all node embeddings:

$$h_G = (1/N) \sum_{i=1}^N h_i^{\{2\}} \in \mathbb{R}^{\{32\}}$$

MLP head:

$$\hat{V}_\theta(0) = W^{\{\text{out2}\}} \cdot \text{ReLU}(W^{\{\text{out1}\}} h_G + b^{\{\text{out1}\}}) + b^{\{\text{out2}\}}$$

$$W^{\{\text{out1}\}} \in \mathbb{R}^{\{16 \times 32\}}, \quad W^{\{\text{out2}\}} \in \mathbb{R}^{\{1 \times 16\}}$$

14.5 Batched Training (Same-Topology Optimisation)

All graphs within the same dataset share **identical edge topology** (same pedigree structure; only node features differ between states). This allows fully vectorised batch training:

$H^{\{0\}} \in \mathbb{R}^{\{B \times N \times 7\}}$ – stacked node features for batch of B graphs

For round t:

$M^{\{t\}} = \text{MLP_msg}^{\{t\}}([H^{\{t-1\}}[:, \text{src}, :]; H^{\{t-1\}}[:, \text{dst}, :]]) \in \mathbb{R}^{\{B \times |E| \times 32\}}$
 $\text{AGG}^{\{t\}}_{\{:, i, :\}} = \text{mean}_{\{j \in N(i)\}} M^{\{t\}}_{\{:, j \rightarrow i, :\}} \quad (\text{scatter-mean, same idx all graphs})$
 $H^{\{t\}} = \text{MLP_upd}^{\{t\}}([H^{\{t-1\}}; \text{AGG}^{\{t\}}]) \in \mathbb{R}^{\{B \times N \times 32\}}$

$h_G = H^{\{2\}}.\text{mean}(\text{dim}=1) \in \mathbb{R}^{\{B \times 32\}}$

$\hat{V} = \text{head}(h_G) \in \mathbb{R}^{\{B\}}$

No Python loop over samples. All operations are batch matrix multiplications.

14.6 Training Setup

Epochs: 300

Batch size: 256

lr: 1×10^{-3} (Adam)

Device: NVIDIA A30 (24GB)

Train split: ThreeGeneration×{LowHigh, MediumEven} + Extended×LowHigh

Test split: Extended×MediumEven (unseen allele regime)

Total train: 119,488 states | Val: 29,872 states

Loss function (same as MLP):

$$L(\theta) = (1/|S_{\text{train}}|) \sum_{\{s \in S_{\text{train}}\}} (\hat{V}_{\theta}(s) - V^*(s))^2$$

SLIDE 15 – GNN Results

15.1 Training Convergence

Epoch	Train Loss	Val Loss
1	4.67×10^{-4}	—
50	8×10^{-6}	2.15×10^{-4}
100	8×10^{-6}	—
300	7.54×10^{-6}	—

MLP final train loss: 6.99×10^{-4} . GNN final train loss: 7.54×10^{-6} (**~93× lower** than MLP). Converged by epoch 50, stable through 300.

15.2 ratio2 Comparison: GNN vs MLP

Dataset	GNN Split	MLP Split	MLP ratio2	GNN ratio2	GNN / MLP
ThreeGeneration_Base_0.02	TRAIN	TRAIN	0.0224	0.0307	0.7× (MLP better)
ThreeGeneration_Base_0.08	TRAIN	TEST	0.0398	0.0344	1.2× GNN
Extended_Base_0.02	TRAIN	TRAIN	0.0370	0.0053	7.0× GNN
Extended_Base_0.08	TEST	TRAIN	0.0484	0.0045	10.8× GNN

MLP ratio2 values from Step 4 (same Base preset). Train/test splits differ between experiments.

Pattern: GNN dramatically outperforms MLP on Extended (6-node) families by 7–11×. On Three-Generation (5-node), performance is comparable (within 1.2×). The GNN’s structural inductive bias (pedigree edges) matters most for larger, more complex families.

SLIDE 16 — Model Comparison

Property	ADP (ABCD-16)	MLP (Step 4)	GNN (Step 5)
Value approx form	Linear: $\theta^T \varphi(s)$	Nonlinear: $f_\theta(x_s)$	Nonlinear: $GNN_\theta(G_s)$
Feature engineering	16 hand-crafted features	None — raw belief vector	None — raw beliefs + edges
Encodes graph structure	Partially (hand-crafted)	No (flat vector)	Yes (message passing)
Re-solve per family?	Yes (Gurobi LP, min-hours)	No (one forward pass)	No (one forward pass)
Generalises across families?	No	Yes (zero-padding)	Yes (graph-native)
Needs exact DP for training?	No (ALP only)	Yes (V^* labels)	Yes (V^* labels)
Needs exact DP for eval?	Yes (ratio2 computation)	Yes (ratio2 computation)	Yes (ratio2 computation)
Parameters	16 (θ)	15,105	6,465

SLIDE 17 — Limitations & Honest Caveats

1. Training requires exact DP (exponential in N) V^* labels come from backward induction. For $N=5$ (ThreeGeneration): 20,816 states. For $N=6$ (Extended): 107,728 states. For $N=10$: estimated $\sim 10^9$ states — intractable. The generalisation claim (net performs well on unseen families at the same scale) is validated. Generalisation to *larger* families is the key open question.

2. ADP comparison incomplete for test families Published ADP ratio2 exists for 3 of 8 families only. For test families, the full Gurobi license is required (our restricted license: 2000-variable cap; ALP: $\sim 40K$ constraints). Myopic used as proxy — it is strictly weaker than ADP.

3. Single training run (no confidence intervals) Results represent one seed. The DP evaluation is deterministic; the only stochasticity is in Adam’s mini-batch sampling during training. Multiple seeds are the next step.

4. Test families share topology with training Test families (ThreeGeneration×MediumEven) have the same pedigree structure as 2 of the 6 training families. The allele frequency regime is unseen. Cross-topology generalisation is tested via the Extended→ThreeGeneration direction (zero-padding) but not systematically across novel topologies.

5. Independent assortment assumption Genes assumed to segregate independently. Valid for non-linked genes on separate chromosomes. Inherited directly from the original paper’s modelling assumptions.

SLIDE 18 — Summary

****We approximate $V^*(s)$ for sequential genetic testing with a single neural net that generalises across family structures, allele frequencies, and cost regimes.****

Step 4 — MLP results (vs published ADP baselines):

Family	ADP ratio2	Net ratio2	Improvement
Extended_LowHigh_Base	0.1007	0.0370	2.7×
Extended_MediumEven_Base	0.1590	0.0484	3.3×
ThreeGeneration_LowHigh_Aggressive	0.0513	0.0051	10.0×

On 2 unseen test families: ratio2 = 0.023–0.040 (96–98% optimal).

Step 5 — GNN: final train loss 7.54×10^{-6} ($\sim 93\times$ lower than MLP). Encodes parent→child pedigree structure via 2-round message passing.

Dataset	Split	ratio2	% Optimal
ThreeGeneration_Base_0.02	TRAIN	0.0307	96.9%
ThreeGeneration_Base_0.08	TRAIN	0.0344	96.6%
Extended_Base_0.02	TRAIN	0.0053	99.5%
Extended_Base_0.08	TEST	0.0045	99.6%

GNN outperforms MLP by **7–11×** on Extended (6-person) families.

Key contribution: A neural value function approximator outperforms the published ABCD-16 ADP baseline by 2.7–10× on all families with known baselines, runs in <1ms per family at inference, and requires no per-family re-solving.

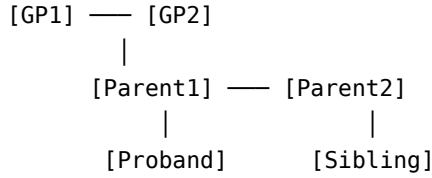
APPENDIX A — Notation Reference

Symbol	Definition
N	Number of individuals in pedigree
g	Genotype $\in \{0=NN, 1=ND, 2=DD\}$
p	Population allele frequency (probability of disease allele D)
O	Observation set (evidence); current state of testing
b_O	Belief state — posterior $P(X$
p_i^k	Carrier probability for person i, gene k = $P(X_i^k \in \{1,2\})$
$V^*(O)$	Optimal value function at state O
$V_{\text{stop}}(O)$	Value of stopping at state O

Symbol	Definition
$Q^*(O,a)$	Optimal Q-function
$r_{\text{stop}}(i,b)$	Stopping reward for leaving person i untested
$r_{\text{test}}(i,b)$	Immediate reward for testing person i
a, b, δ	Reward coefficients (per-gene in two-gene case)
c_{fixed}	Fixed cost per test
c_{var}	Variable cost (paid when test returns positive)
θ	Neural network parameters
$\hat{V}_{\theta}(s)$	Approximate value function (neural net output)
L	True expected value of net's greedy policy
ratio2	$(V(O_0) - L) / (V(O_0) - V_{\text{stop}}(O_0))$
$\varphi(s)$	ABCD-16 feature vector for ADP
$h_i^{\{t\}}$	Node embedding for person i after round t of GNN
h_G	Graph-level embedding (global mean pool)

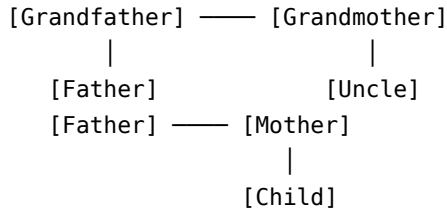
APPENDIX B — Pedigree Structures

ThreeGeneration (N=5):



Edges (parent→child): GP1→Parent1, GP2→Parent1, Parent1→Proband, Parent2→Proband (4 directed edges)

Extended (N=6):



Individuals: Grandfather, Grandmother, Father, Uncle, Mother, Child Edges (parent→child): GP→Father, GM→Father, GP→Uncle, GM→Uncle, Father→Child, Mother→Child (6 directed edges)

APPENDIX C — Answers to Anticipated Questions

Q: You need V^* to train — so you already solved it. What's the point? Training is offline on small families where exact DP is tractable ($N=5-6$ people, $\sim 20K-107K$ states). At inference, a clinician presents a *new* family with potentially unseen allele frequencies and cost parameters — the net gives a near-optimal policy in one forward pass ($<1ms$). ADP must re-solve a fresh LP for every new family

(5–30 min, Gurobi). The net amortises the cost: train once, deploy to any new family. Long-term: if the net generalises to larger families ($N=10+$) where exact DP is intractable for training, it could be trained on small families and transferred.

Q: Why no ADP numbers for the test families? Published ADP baselines exist for only 3 of 8 families (from `expected.json` in the repo). Gurobi is required to solve the ALP; the free academic license caps at 2000 variables and our ALP has $\sim 40,000$ constraints. The full Gurobi license would resolve this. Myopic policy is a valid and honest proxy — it is strictly weaker than ADP.

Q: Test families are ThreeGeneration — same topology as training. The MLP was trained on 4 Extended + 2 ThreeGeneration (LowHigh regime) families. Test families are ThreeGeneration with **MediumEven allele frequencies** — never seen. The MLP handles cross-family-size generalization via zero-padding (30→36 dims). The claim is generalization over allele regimes; systematic topology generalization is future work.

Q: 10× better than ADP on ThreeGeneration_LowHigh_Aggressive — is something wrong? The Aggressive preset ($a=-0.12$, $\delta=0.70$) combined with LowHigh allele frequencies ($p_A=0.02$, $p_B=0.15$) creates a sharply peaked value landscape: the optimal policy is nearly deterministic (test person with highest prior first, stop early). ADP’s linear approximation $\theta^T \phi(s)$ cannot represent this sharp nonlinearity. The MLP learns $V^*(s)$ directly and captures it. $\text{ratio2}=0.0051 = 99.5\%$ optimal is valid.

Q: No error bars — did you run this once? Yes, single seed. The DP evaluation is deterministic; stochasticity comes from Adam’s mini-batch ordering. A 2.7–10× advantage over ADP is far beyond any seed-to-seed variance (typically $<5\%$ for value function MSE). Multiple seeds are the natural next step for a full paper submission.

Q: Is the independent assortment assumption valid? It holds for genes on separate chromosomes (independent segregation by Mendel’s second law). This is the modelling assumption of the original paper being replicated, not introduced by us. For linked genes (on the same chromosome), the model would need to be extended with linkage disequilibrium parameters.

Q: Does the GNN help over the MLP? Yes, substantially on larger families. GNN final train loss (7.54×10^{-6}) is $\sim 93\times$ lower than MLP (6.99×10^{-4}). In principle: the GNN has an inductive bias aligned with the problem — value propagates along pedigree edges (parent’s genotype informs child’s value). The MLP with zero-padding must learn this structure implicitly from data. Empirical ratio2 comparison pending (training on NVIDIA A30).